

SENSORES, ATUADORES E ARMAZENAMENTO

TEMA 1 – SENSORES DE GÁS

O sensor de gás, ou melhor dizendo, os sensores de gás têm uma particularidade especial, pois existem diversos tipos de gases que podem ser detectados. A família de sensores mais famosa para a detecção de gás é a MQ. São do tipo eletrocatalítico, modernos e de baixo custo. Seu material é composto por uma bobina de fio de platina aquecido eletricamente, dentro de uma base de cerâmica. Repare na Figura 1, em que a bobina está envolvida pela base de cerâmica.

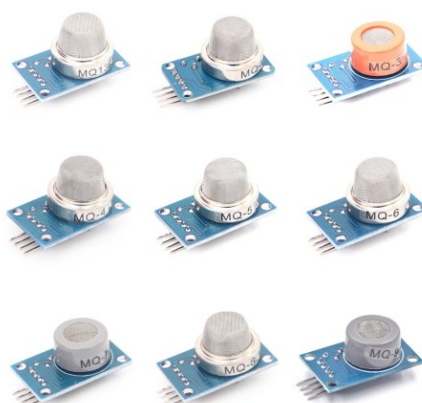
Figura 1 – Parte interna do sensor MQ-135



Fonte: Candido, 2017.

Para facilitar, o sensor foi acoplado a uma pequena placa com o circuito necessário para um bom funcionamento e um circuito comparador. A Figura 2 representa alguns sensores da família MQ.

Figura 2 – Representação de alguns sensores da família MQ



Fonte: Candido, 2017.

A Figura 2 não mostra todos os sensores dessa grande família. Para facilitar o acesso à informação, e consequentemente a escolha certa do sensor, observe a Tabela 1.

Tabela 1 – Família MQ

Sensor	Sensibilidade	Tensão aquecimento
MQ-2	Metano, butano, gás GLP, CO, fumo.	5v
MQ-3	Álcool, etanol, fumo.	5v
MQ-4	Metano, gás CNC.	5v
MQ-5	Gás GLP.	5v
MQ-6	Gás GLP, butano.	5v
MQ-7	Monóxido de carbono.	Tensão alternada de 5 e 1.4v
MQ-8	Gás hidrogênio.	5v
MQ-9	Monóxido de carbono, gases inflamáveis.	Tensão alternada de 5 e 1.5v
MQ-131	Ozônio.	6v
MQ-135	Benzeno, álcool, fumo.	5v
MQ-136	Sulfeto de hidrogênio.	5v
MQ-137	Amoníaco	5v
MQ-138	Benzeno, tolueno, álcool, acetona, propano, gás formaldeído, gás hidrogênio.	5v
MQ-214	Metano, gás natural.	6v
MQ-216	Gás natural, gás de carvão.	6v
MQ-303A	Álcool, etanol e fumo.	0.9v
MQ-306A	GLP, gás butano.	0.9v
MQ-307A	Monóxido de carbono.	Tensão alternada de 0.2 e 0.9v
MQ-309A	Monóxido de carbono, gases inflamáveis.	Tensão alternada de 0.2 e 0.9v

A Tabela 1 traz o nome do sensor, o tipo(s) de gás(es) que ele detecta e a tensão necessária para o aquecimento. Com essa informação em mãos, é só ver qual será o seu projeto e comprar o sensor correto. A programação dos sensores da família MQ é bem semelhante entre eles, mudando apenas algumas poucas particularidades de cada sensor.

A Tabela 2 é uma fonte de consulta e pesquisa, quando necessário.

Tabela 2 – Aspectos técnicos de cada sensor

Sensor	Datasheet
MQ-2	http://www.haoyuelectronics.com/Attachment/MQ-2/MQ-2.pdf
MQ-3	https://www.sparkfun.com/datasheets/Sensors/MQ-3.pdf
MQ-4	https://www.pololu.com/file/0J311/MQ4.pdf
MQ-5	https://www.parallax.com/sites/default/files/downloads/605-00009-MQ-5-Datasheet.pdf
MQ-6	https://www.sparkfun.com/datasheets/Sensors/Biometric/MQ-6.pdf
MQ-7	https://www.sparkfun.com/datasheets/Sensors/Biometric/MQ-7.pdf
MQ-8	https://dlnmh9ip6v2uc.cloudfront.net/datasheets/Sensors/Biometric/MQ-8.pdf
MQ-9	https://www.pololu.com/file/download/MQ9.pdf?file_id=0J314
MQ-131	http://www.gassensor.ru/data/files/ozone/MQ-131-O3.pdf
MQ-135	https://www.olimex.com/Products/Components/Sensors/SNS-MQ135/resources/SNS-MQ135.pdf
MQ-136	https://www.mysensors.org/dl/57c3eb071cb0e34c90057a/design/MQ-136.pdf
MQ-137	http://eph.ccs.miami.edu/precise/GasSensorSpecs/NH3.pdf
MQ-138	https://www.mysensors.org/dl/57c3eb071cb0e34c90057a/design/MQ-138.pdf
MQ-214	https://www.mysensors.org/dl/57c3eb071cb0e34c90057a/design/1341.pdf
MQ-216	http://www.china-total.com/Product/meter/gas-sensor/MQ-216.pdf
MQ-303A	http://www.kosmodrom.com.ua/pdf/MQ303A.pdf
MQ-306A	http://shop.aftabrayaneh.com/image/data/product%20extension/MQ/46.pdf
MQ-307A	http://www.gassensor.ru/data/files/carbon_monoxide/MQ307A.pdf
MQ-309A	http://www.sensorica.ru/pdf/MQ-309A.pdf

TEMA 2 – SENSOR DE GÁS – MQ-2

Como vimos, a família MQ é grande, por isso foi escolhido o sensor MQ-2 para realizar o projeto. Com a IoT atuando na automatização de residências, o sensor MQ-2 é o mais utilizado, pois ele detecta o gás butano e o GLP, que são utilizados nas residências e também detectam fumaça. Observe a representação do sensor na Figura 3.

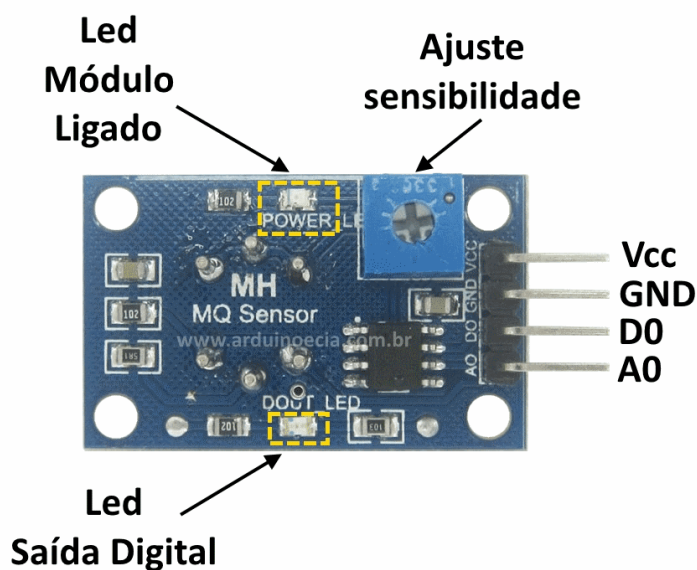
Figura 3 – Sensor MQ-2



Fonte: Mind Ocean/Shutterstock.

Note na Figura 3 que o sensor está acoplado há um pequeno circuito. Na parte de trás, repare que o circuito possui em um pequeno retângulo azul. Repare na Figura 4.

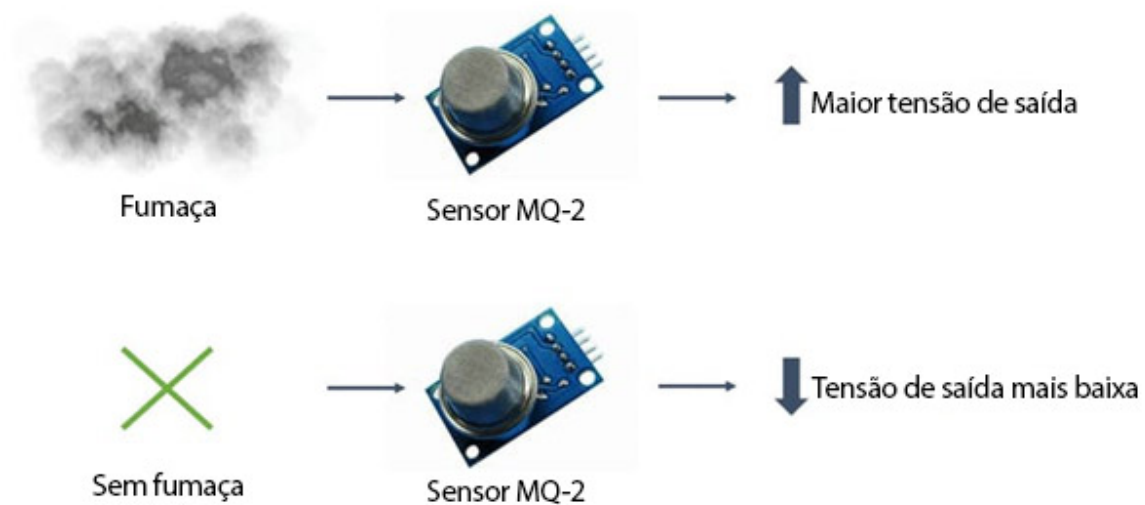
Figura 4 – Parte de trás do sensor MQ-2



Fonte: Alarme sensor..., 2015.

Esse retângulo é um potenciômetro que tem a função de ajustar a sensibilidade do sensor. Note também que o sensor apresenta duas saídas de comunicação: uma saída analógica (A0) e uma saída digital (D0). A saída analógica pode ser utilizada para fazer a medição do nível de concentração de gases detectados. Quanto maior a concentração, maior vai ser o nível do sinal na saída analógica. A saída digital pode ser utilizada para abrir ou fechar o fornecimento de gás ou mesmo para tocar uma sirene. A Figura 5 foi retirada da documentação oficial do site do Arduino e representa a saída analógica do sensor.

Figura 5 – Parte de trás do sensor MQ-2



Fonte: Alarme sensor..., 2015.

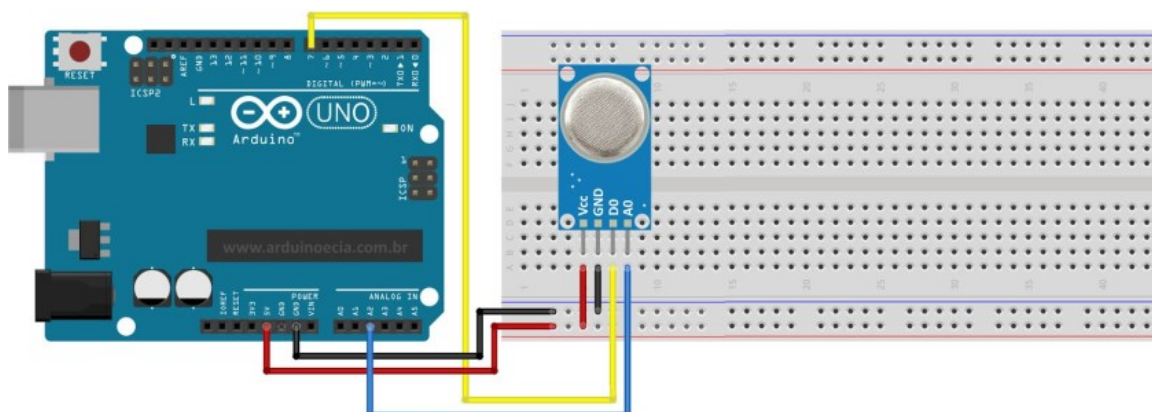
2.1 Projeto MQ-2

O projeto a seguir utiliza um sensor de gás MQ-2 e vai simplesmente emitir um alerta no monitor serial se o ambiente está ou não com algum tipo de gás (metano, butano, gás GLP, CO, fumo). Para a realização desse projeto, será necessário:

- Arduino
- Sensor MQ-2
- Jumper's
- Protoboard

O primeiro passo é fazer a montagem do hardware do projeto. A Figura 6 representa as ligações necessárias para o projeto.

Figura 6 – Representação da montagem do Projeto MQ-2



Repare na Figura 6 que o sensor MQ-2 não precisa de um resistor, isso ocorre, pois, esse sensor trabalha com a tensão de 5V, a mesma da placa do Arduino. A saída digital foi conectada com o jumper amarelo na porta digital 7 e a saída analógica foi conectada com o jumper azul na porta analógica A2. Após a montagem do hardware, é preciso criar sketch. Observe a figura 7.

Figura 7 – Representação do sketch do Projeto MQ-2

```
1  #define analogica A2
2  #define digital 7
3
4  int valor_analogico;
5  int valor_digital;
6
7  void setup() {
8      Serial.begin(9600);
9      pinMode(analogica, INPUT);
10     pinMode(digital, INPUT);
11 }
12
13 void loop() {
14     valor_analogico = analogRead(analogica);
15     valor_digital = digitalRead(digital);
16
17     Serial.print(valor_analogico);
18     if(valor_digital == 0) Serial.println("GAS DETECTADO !!!");
19     else Serial.println("GAS AUSENTE !!!");
20     delay(1000);
21 }
```

A Figura 7 representa o sketch e vai escrever na saída serial se foi ou não detectado algum gás. Essa verificação se dará de 1 em 1 segundo.

TEMA 3 – BIBLIOTECAS

Uma biblioteca nada mais é que um pedaço de código que possui uma funcionalidade específica dentro de um sketch. O uso das bibliotecas simplifica para o usuário na hora de desenvolver uma aplicação, pois a biblioteca já é um trecho de código pronto, ficando para o usuário incorporar (ou chamar) a biblioteca no sketch. Como exemplo, vejamos: pensando em um acelerômetro MPU6050 e sua biblioteca MPU6050.h. Essa biblioteca tem funções desenvolvidas especificamente para executar tarefas como, configurar o acelerômetro, ler dados de aceleração, giroscópio, temperatura etc.

Além de facilitar a programação, o uso das bibliotecas deixa o código mais simples e mais limpo. Utilizando a MPU6050 ainda como exemplo, ela apresenta cerca de 4000 linhas de programação, sem contar que dentro dela existem outras bibliotecas sendo chamadas. O usuário não precisa conhecer as 4000 linhas e o que cada uma delas faz, apenas precisa saber quais são os comandos usados para chamar determinadas funções dentro da biblioteca e o que elas retornam.

Para compreender como uma biblioteca pode ser criada é preciso saber que basicamente uma biblioteca é dividida em 3 partes/arquivos:

- **Header:** é o cabeçalho, possui a extensão .h e contém as funções e variáveis da biblioteca;
- **Source:** é o arquivo fonte, possui a extensão .cpp e contém toda a lógica e a programação em si das funções;
- **Keywords:** é um arquivo .txt, específico da IDE do Arduino e sua função é padronizar a cor dos comandos na escrita do código.

O arquivo keywords.txt não é obrigatório, mas é interessante, pois como as funções estão sendo criadas, o programador pode definir uma coloração diferente para cada função.

3.1 Tipos de bibliotecas

As bibliotecas podem ser classificadas em 3 categorias: core, padrão, adicionais. Algumas bibliografias apontam 2 tipos de bibliotecas (padrão e adicionais) deixando o tipo core de fora da classificação. Repare como é composta essa classificação:

- **Core:** é a biblioteca que já vem na IDE do Arduino. Sem ela nenhuma programação poderia ser realizada. As funções `digitalRead`, `analogRead`, `Serial.begin`, entre outras, fazem parte dessa biblioteca.
- **Padrão:** são bibliotecas que vem junto na instalação da IDE, mas elas não vão estar relacionadas com o seu projeto de forma automática, pois o Arduino possui uma memória limitada. Elas ficam disponíveis e o programador pode fazer a chamada delas quantas vezes for necessário.
- **Adicionais:** são bibliotecas desenvolvidas por terceiros (qualquer programador) e podem ser importadas (instaladas) para a sua IDE. Geralmente elas fornecem funcionalidades que não são encontradas nas bibliotecas da categoria padrão. Com a utilização das bibliotecas adicionais, o uso do Arduino fica praticamente ilimitado.

Para esse material, o foco são as bibliotecas do tipo padrão. Para incluir uma biblioteca padrão, é utilizado o comando `#include` sempre no início do programa (sketch). Imagine que você quer fazer um projeto que vai utilizar uma conexão WI-FI, e que para isso funcionar é preciso um shield (hardware) WI-FI que vai ser acoplado ao Arduino. Para tal projeto é necessário a biblioteca padrão WI-FI, que deve ser incluída no início do projeto. Repare na Figura 8.

Figura 8 – Comando para incluir a biblioteca WI-FI

```
#include <Stepper.h>
```

Após o comando `#include`, o nome da biblioteca padrão deve vir entre os símbolos `<` e `>` e sempre finalizar com a extensão `.h`. Note também que nessa linha de comando não existe a finalização com ponto-e-vírgula. Quando uma biblioteca é incluída, todas as suas funções ficam disponíveis, dentro desse programa, para o programador utilizar. Para finalizar o estudo das bibliotecas, repare no Quadro 2, que representa as bibliotecas do tipo padrão, traduzidas do site oficial do Arduino.

Quadro 1 – Bibliotecas do tipo padrão

EEPROM	Usada para ler e gravar dados em uma memória EEPROM no Arduino. No Uno a EEPROM tem o tamanho de 1024 bytes e no Mega, de 4096 bytes.
Ethernet	Permite conectar o Arduino à Internet ou à rede local usando um shield Ethernet.
Firmata	Esta biblioteca permite a comunicação entre o Arduino e aplicações em um computador via protocolo de comunicação serial.
GSM	Conectar a uma rede GSM/GPRS usando um shield GSM.
LiquidCrystal	Com essa biblioteca podemos controlar displays de cristal líquido (LCD).
SD	Biblioteca muito importante, usada para que seja possível escrever e ler dados em cartões de memória SD/SDHC.
Servo	Controlar motores servo.
SPI	Comunicação com dispositivos usando o barramento SPI (Serial Peripheral Interface).
SoftwareSerial	Permite a comunicação serial usando os pinos digitais da placa.
Stepper	Controlar motores de passo.
TFT	Permite desenhar imagens e formas e escrever texto em uma tela TFT.
WI-FI	Permite conexão à rede local e Internet por meio de um shield WI-FI.
Wire	Biblioteca que permite enviar e receber dados por meio de uma interface TWI/I2C (interface a dois os) em uma rede de dispositivos ou sensores.

Fonte: Elaborado com base em Reis, 2015.

TEMA 4 – SHIELDS

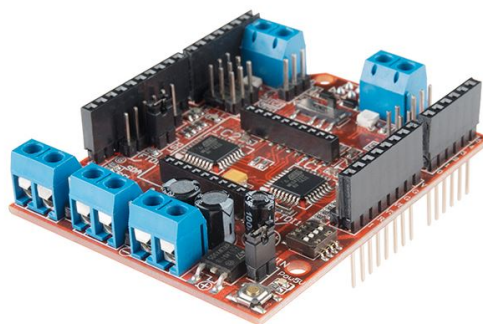
No tema anterior, tratamos sobre as bibliotecas, e que elas podem expandir algumas funcionalidades do Arduino. Nesse tema, abordamos os shields, que também têm a função de aumentar a funcionalidade que um Arduino pode desempenhar, mas agora em forma de outras placas (hardware). No mercado,

existem diversos shields que expandem as mais diversas funcionalidades. As mais comuns são:

- Shields de motor
- Shield de Ethernet
- Shield WI-FI
- USB Host Shield
- Shield microSD/SD
- Shield MP3
- Shield LCD TFT

Para exemplificar um tipo de shield e suas características, foi escolhido o shield de motor. Sua função é controlar motores DC, servomotores ou motores de passo, podendo ser utilizado na criação de protótipos de robôs, carrinhos ou qualquer outro projeto que envolva motores. Várias empresas produzem shields de motor. A Figura 9 representa o shield da empresa Sparkfun.

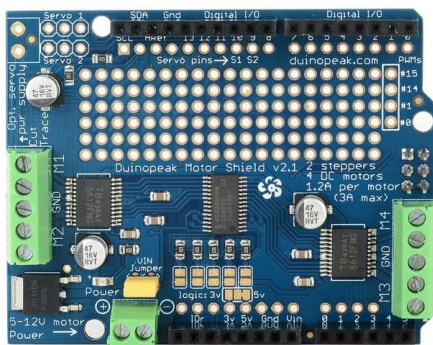
Figura 9 – Motor Shield ComMotion da Sparkfun para Arduino



Fonte: Reis, 2015.

Repare na Figura 10, que representa o shield de motor da empresa Duinopeak.

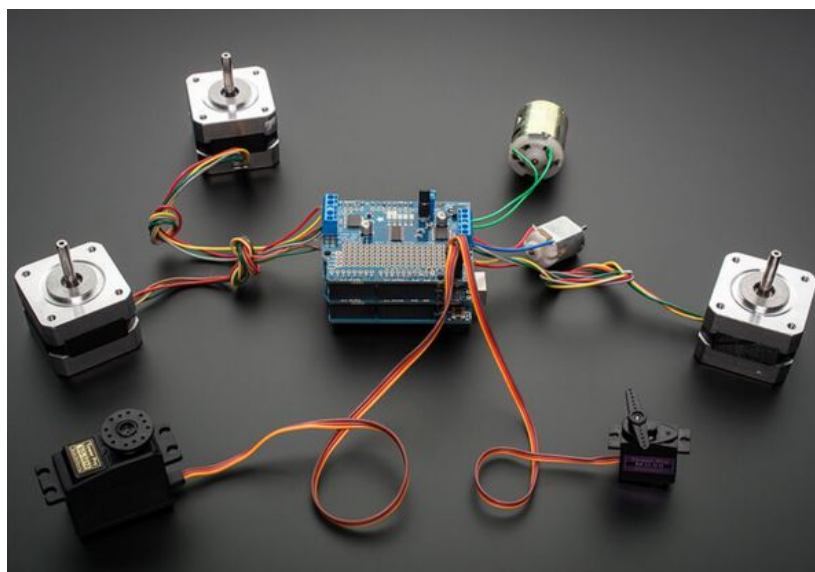
Figura 10 – Motor Shield Duinopeak



Fonte: Reis, 2015.

Esse shield permite controlar até 2 motores de passo e 4 motores DC. Observe na Figura 11 as várias ligações realizadas.

Figura 11 – Motor Shield Duinopeak com vários motores conectados



Fonte: Reis, 2015.

Repare na Figura 11 que no hardware central existem 3 placas. O primeiro de baixo é a placa do Arduino, e as duas de cima são shields de motor, conectados entre si, ampliando sua capacidade.

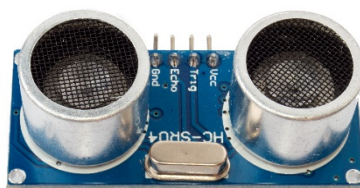
Sempre na utilização de qualquer shield, é necessário atenção na conexão física entre o shield e a placa do Arduino, principalmente na elétrica, e fazer a importação da biblioteca para que shield funcione corretamente. Para

conhecimento geral, existem mais de 500 shields, sendo possível obter maiores informações em uma lista oficial¹.

TEMA 5 – SENSOR ULTRASSÔNICO DE DISTÂNCIA

O sensor ultrassônico é muito utilizado na área da robótica, pois ele pode ser usado desde a medição distâncias até processos para evitar colisões, pois pode medir distâncias que variam de 20mm a 20 metros, com erro de medição de no máximo 1%. Uma aplicação bem simples desse sensor é a sua utilização nos veículos, principalmente na ré. Os carros utilizam um sistema de sensor ultrassônico que consegue medir a distância entre o carro e algum outro objeto (outro carro, parede, lata de lixo etc.). O sensor que vai ser trabalhado nesse material é o HC-SR04. Observe a representação do sensor na Figura 12.

Figura 12 – Sensor Ultrassônico HC-SR04



Fonte: Pozdeyev Vitaly/Shutterstock.

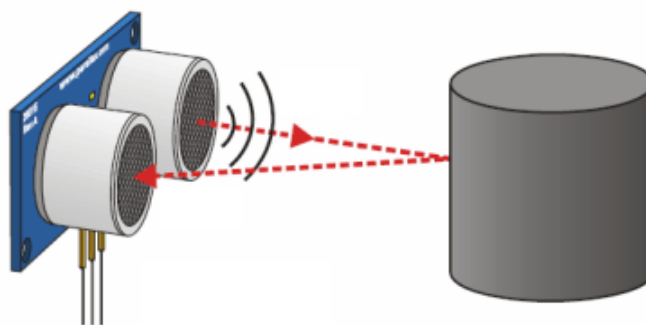
Note na Figura 12 que o sensor apresenta duas cavidades. Uma delas emite e a outra recebe um pulso sonoro. Mota (2018) descreve o funcionamento do sensor ultrassônico HC-SR04:

Tudo começa pela emissão de um pequeno pulso sonoro de alta frequência que se propagará na velocidade do som no meio em questão. Quando este pulso atingir um objeto, um sinal de eco será refletido para o sensor. A distância entre o sensor e o objeto pode então ser calculada caso saibamos o tempo entre a emissão e a recepção do sinal, além da velocidade do som no meio em questão.

Observe a Figura 13, que representa a emissão e retorno do pulso sonoro.

¹ <<http://shieldlist.org>>.

Figura 13 – Pulso sensor ultrassônico HC-SR04



Fonte: Mota, 2018.

O sensor ultrassônico também é muito utilizado na indústria², pois ele pode medir variáveis como enchimento, curvatura e altura sem a necessidade de nenhum contato.

5.1 Projeto HC-SR04

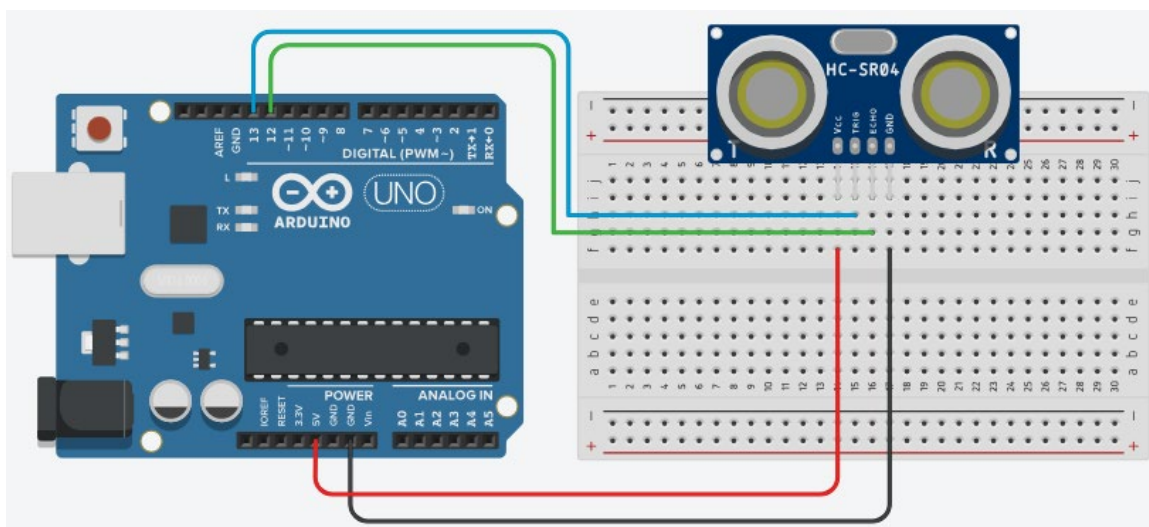
O projeto a ser desenvolvido vai ser utilizado para medir a distância entre dois objetos e mostrar o valor da distância no monitor serial. Para a realização desse projeto será necessário:

- Arduino
- Sensor HC-SR04
- Protoboard
- Jumper's

O primeiro passo é fazer a montagem do hardware do projeto. A Figura 14 representa as ligações necessárias para o projeto.

² <<https://www.citisystems.com.br/sensor-ultrassonico/>>.

Figura 14 – Representação da montagem do Projeto HC-SR04



Repare na Figura 14 que o primeiro pino (jumper vermelho) está conectado à alimentação de 5v e o último pino (jumper preto) está conectado no GND. O pino “trig” está conectado com o jumper azul na porta digital 13 que deve ser configurada como saída. E por último, o pino “echo” está conectado na porta digital 12 e deve ser configurada como entrada no sketch. Após a montagem do hardware, é preciso programar (criar o software).

Figura 15 – Representação do sketch do Projeto HC-SR04

```
1 // Projeto Sensor HC-SR04
2
3 #include <Ultrasonic.h> // adiciona a biblioteca
4 Ultrasonic ultrassom(13,12);
5 // define o nome do sensor(ultrassom) e pinos trig(13) e echo(12)
6
7 long distancia;
8
9 void setup() {
10     Serial.begin(9600);
11 }
12
13 void loop()
14 {
15     distancia = ultrassom.Ranging(CM);
16     // retorna a distância em centímetros(CM) ou polegadas(INC)
17     Serial.print(distancia); //imprime o valor da variável distancia
18     Serial.println("cm");
19     delay(1000);
20 }
```

Repare na linha 3 do sketch representado na Figura 15, onde é feita a inclusão da biblioteca Ultrasonic.h, responsável por conter as funções necessárias para o funcionamento do sensor ultrassônico HC-SR04. Na linha 4, há uma

chamada de uma função (que está dentro da biblioteca) onde é definido o nome do sensor (utilizado na linha 15) e passados dois parâmetros:

- O primeiro parâmetro informa qual a entrada digital que o pino trig do sensor foi conectado;
- O segundo parâmetro informa qual a entrada digital que o pino echo do sensor foi conectado;

Na linha 15 a variável *distancia*, que foi criada na linha 7, recebe o retorno da função “Ranging”. Lembre-se que o sensor recebeu o nome de “ultrassom” na linha 4 e esse nome é utilizado na chamada da função “Ranging”. Como o retorno da função vai ser em centímetros, o parâmetro passado foi CM. Nas linhas 17 e 18 está a saída (impressão) do valor armazenado na variável *distancia* para o monitor serial. Essa medição se repete de 1 em 1 segundo.

REFERÊNCIAS

ARDUINO E CIA. **Alarme sensor de gás com o módulo MQ-2**, 2015. Disponível em: <<https://www.arduinoecia.com.br/2015/01/alarme-sensor-de-gas-modulo-mq-2.html>>. Acesso em: 3 jun. 2019.

CANDIDO, G. Sensor de Gás MQ-135 e a família MQ de detectores de Gás. **Vida de Silício**, 28 set. 2017. Disponível em: <<https://portal.vidadesilicio.com.br/sensor-de-gas-mq-135/>>. Acesso em: 3 jun. 2019.

MOTA, A. HC-SR04 – Sensor ultrassônico de distância com Arduino. **Vida de Silício**, 14 maio 2017. Disponível em: <<https://portal.vidadesilicio.com.br/hc-sr04-sensor-ultrassonico/>>. Acesso em: 3 jun. 2019.

REIS, F. dos. Arduino: Conhecendo os shields. **Bosontreinamentos**, 2015. Disponível em: <<http://www.bosontreinamentos.com.br/eletronica/arduino/arduino-conhecendo-os-shields/>>. Acesso em: 3 jun. 2019.

STEVAN JUNIOR, S. L. **Automação e instrumentação industrial com Arduino: teoria e projetos**. São Paulo: Érica, 2015.